

```
import face_recognition
import cv2
import numpy as np

# This is a demo of running face recognition on live video from
# other example, but it includes some basic performance tweaks
# 1. Process each video frame at 1/4 resolution (though still
# 2. Only detect faces in every other frame of video.

# PLEASE NOTE: This example requires OpenCV (the 'cv2' library)
# OpenCV is 'not' required to use the face_recognition library.
# specific demo. If you have trouble installing it, try any of

# Get a reference to webcam #0 (the default one)
video_capture = cv2.VideoCapture(0)
```

Fig. 10: Python code for face recognition

and numpy, as shown in Fig. 10. Create different arrays for recognising faces and names. Make sure to include the image file names of all known persons (who you want to be recognised) in the code and store them in a folder for correct face recognition (refer Fig. 11).



In the next part of the code, the program matches the face that has been captured by the camera with the array of known faces. If the face matches, the code will run the 'espeak.synth ()' synthesiser function to call out the person's name through the speaker connected to the Raspberry Pi. The code with 'espeak.synth ()' function is shown in Fig. 12.

Connection with speaker and camera

To get audio output through Raspberry Pi, connect TRRS (Aux) audio output of Raspberry Pi to TRRS(Aux) of any speaker with an amplifier. Connect the Raspberry Pi camera module to the camera port present in the Raspberry Pi board.

Now power on the Arduino Nano board connected with the OLED displays via 5V pin of Raspberry Pi. Your face recognition robot is ready to work.

Testing procedure

After hardware connections and software setup are completed, reboot your Raspberry Pi.

Open the face recognition script (FaceRecognitionv1.py) from the Raspberry Pi terminal and run it. You should be able to see the robot's eye movements through the OLED displays.

If you go in front of the camera, the robot will recognise your face. It will call out your name and also display your name on the computer screen, as shown in Fig. 13. If your name is not included in the code, it will display 'unknown' on the screen. **EFY**

Ashwini Kumar Sinha is an electronics hobbyist and tech journalist at EFYi

```
# Family member 1
ashwini_image = face_recognition.load_image_file("ash.jpg")
ashwini_face_encoding = face_recognition.face_encodings(ashwini_image)[0]

# Family member 2
gaurav_image = face_recognition.load_image_file("gaurav.jpg")
gaurav_face_encoding = face_recognition.face_encodings(gaurav_image)[0]

# Create arrays of known face encodings and their names
```

Fig. 11: Include names of known persons in the code

```
face_names.append(name)
espeak.synth(name)

process_this_frame = not process_this_frame

# Display the results
for (top, right, bottom, left), name in zip(face_locations, face_names):
    # Scale back up face locations since the frame we detected in was scaled
    top *= 4
    right *= 4
    bottom *= 4
    left *= 4

    # Draw a box around the face
    cv2.rectangle(frame, (left, top), (right, bottom), (0, 0, 255), 2)

    # Draw a label with a name below the face
    cv2.rectangle(frame, (left, bottom - 35), (right, bottom), (0, 0, 255),
    font = cv2.FONT_HERSHEY_DUPLEX
    cv2.putText(frame, name, (left + 6, bottom - 6), font, 1.0, (255, 255,

# Display the resulting image
cv2.imshow('Video', frame)

# Hit 'q' on the keyboard to quit!
if cv2.waitKey(1) & 0xFF == ord('q'):
    break

# Release handle to the webcam
```

Fig. 12: The code with espeak.synth () function

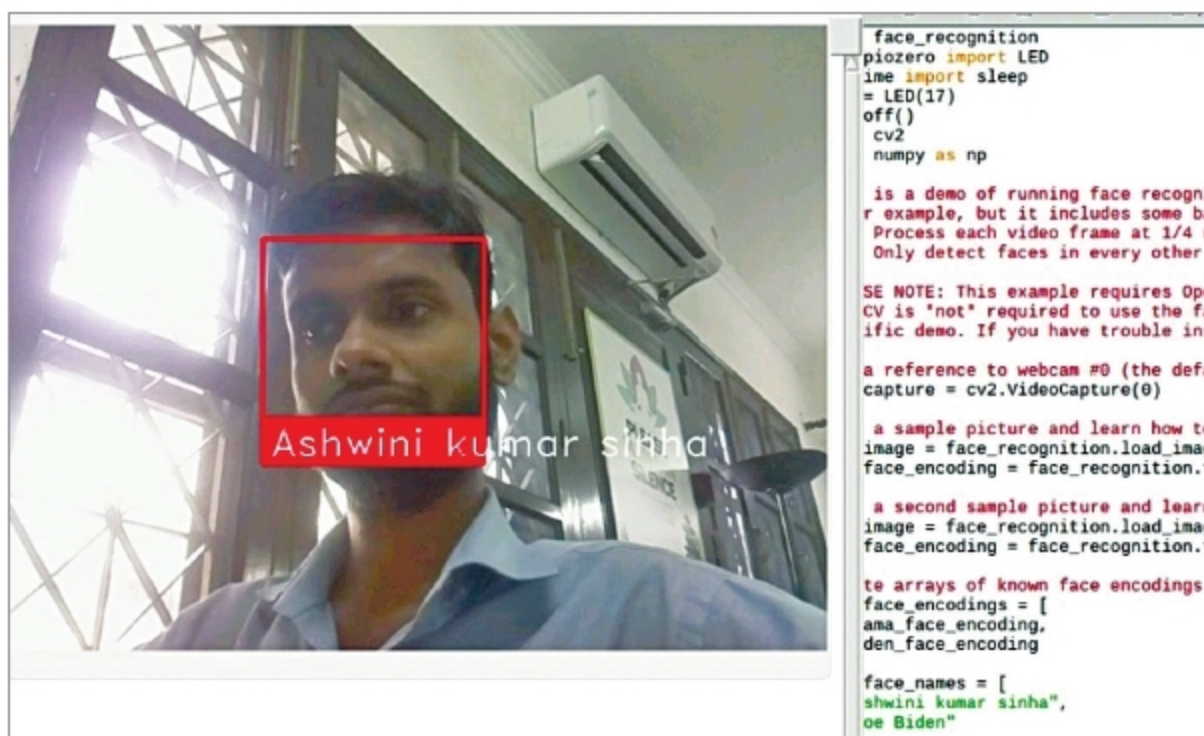


Fig. 13: Face detection with name being displayed on the computer screen